within the context, it is rendered as an empty string. The effect is that errors are handled silently. This is both a feature and a frustration. Blank spots on the page are less problematic for your users, but it may be a bit trickier to find bugs, as the evidence is sneakily silent.

The simplest possible template is text containing no special characters. When rendered, you will get exactly the text you gave the template—what you put in is what you get out. This is of limited value. To have your template change based on the context, Django provides four constructs:

- *Variable rendering via matched double braces*—`{{ name }}`
- *Script actions, like conditionals and loops using tags*—`{% now "Y" %}`
- *Comments*—`{# a comment #}`
- *Output modification via filtering data*—`{{ name | upper }}`

A set of matched braces, sometimes known as *mustache braces*, gets used to display a variable. The name given inside the braces is used as a key in the context dictionary and replaced with the corresponding value.

A variable can be referenced using dot-notation to get at the contents of an object. Consider a `Person` class with a `last_name` attribute. If the context contained an object named `person`, then `{{ person.last_name }}` uses the person object and dereferences the `last_name` attribute on that object.

Tags in the template language get used to perform logic actions inside the template. Some tags get used on their own, and some are paired. The `{% now %}` tag renders as the current date. This tag takes a parameter that indicates the format of the date. Rendering `{% now "Y" %}` results in the current year.

Paired tags typically control the rendering of the block between the tags. One such pair is `{% if %}` and `{% endif %}`, which surround a block that is conditionally rendered. The `{% if %}` tag takes a condition to be evaluated as an argument and behaves similarly to the `if` statement in Python. For example, `{% if num > 3 %} Lots {% endif %}` renders `Lots` if the key `num` exists in the context dictionary, is a number, and has a value greater than 3. A variety of common tags are covered later in this chapter.

Filters are a way of modifying data in a variable. Remember, variables get displayed via paired double braces. You apply a filter to a variable by using the pipe (`|`) operator. For example, `{{ name | upper }}` applies the `| upper` filter to the value contained within `name`. The `| upper` filter is similar to `str.upper()`, returning the contents of `name` in capital letters. Several filters and their use are covered later in this chapter.

There are over 80 tags and filters built into the Django template language. Appendix C is a glossary of the tags and filters built into Django and points you to the online documentation. The template language is also extensible, meaning you can even write your own tags and filters (https://docs.djangoproject.com/en/5.0/howto/custom-template-tags/). Many third-party libraries, written and shared by fellow community members, are available to install as Django apps. For example, the Django Packages website has a listing dedicated to template tag packages: https://djangopackages.org/grids/g/templatetags/.